

Session 3: the hands-on workflow, and the algorithm families

Tuesday 15 September 2026, 11:00 to 12:30 (Montreal time). Videos 30 to 39. Student edition: the questions are here; the answers are discussed in the session.

Questions. Because of the interpretation delay, please send questions in the Zoom chat. Better still: send them afterwards to the WhatsApp number you already have, and they will be answered in writing.

Cleaning data: missing values and encoding

A model learns only from what is on the table: fill the gaps carefully, and turn words into numbers without inventing an order.

The shape of every block: the one idea, a story, three things to keep, three questions, your turn, three things from the block.



V30, page 1 of 19 Cleaning data: missing values and



V30, page 2 of 19 The real bottleneck

The Full Workflow



This order matters. Each step validates the one before it.

V30, page 16 of 19 The full workflow

The one idea

Before a model can learn, the table must be complete and numeric, and every repair is a guess you write down.

A blank you fill with a typical value

3 customers out of 10 have no age.

Fill the three blanks with the middle age of the other seven, 34, and note it in the log.

The model sees a full column and a documented guess.

the word: imputation

A word you turn into a number

Montreal = 1, Toronto = 2, Vancouver = 3 tells the model Vancouver is 'three times' Montreal.

One column per city, holding 0 or 1, tells it nothing false.

Numbers in sequence only where a real order exists.

the word: encoding

Detect, fill, encode, verify, in that order, and write down what you did.

Imputing Numeric Columns

IMPUTE: MEAN

(fast & simple)
sensitive to
OUTLIERS



(mean, ship)
sensitive to
OUTLIERS

IMPUTE: MEDIAN

(uses middle value)
ROBUST to
skewed data



median
ROBUST to
skewed data

Which is robust?

Mean: Mean - sensitive to outliers
Median: Median - insensitive to outliers

1. Mean of outliers is not the same as the mean of the data.

V30, page 6 of 19 Imputing numeric columns

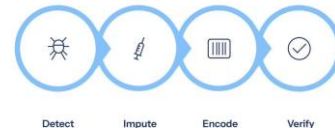
The Label Encoding Trap

- ☒ Label-encode city names
New York = 1, Los Angeles = 2. Your model thinks LA is "bigger".
- ☒ One-hot encode city names
No false rank. Three independent binary signals.

Match the encoding to the truth in your data.

V30, page 15 of 19 The label encoding trap

The Full Workflow



This order matters. Each step validates the one before it.

V30, page 16 of 19 The full workflow

The story

The planes that did not come back

Step 1

New York, 1943

During the Second World War, a group of statisticians at Columbia University was asked where bombers should carry extra armour. The evidence on the table: the pattern of hits on planes that had returned from missions.

Step 2

The pattern

The returning planes showed many hits on the wings and the body, and few around the engines and the cockpit.

Step 3

The obvious answer

Put the armour where the holes are. That is where the planes get hit, so protect those parts.

Step 4

Abraham Wald's question

Wald asked about the planes that were not on the table: the ones that never came back. A clean spot on a survivor is a place where a hit was fatal. The armour belongs where the holes are missing.

Step 5

For our data

A blank cell is not 'nothing'. Before filling it, ask why it is blank. The reason a value is missing can be the most informative thing in the row.

Retold from the Statistical Research Group memoranda (Columbia University, 1943) as summarised in reference articles. The popular version adds drama the records do not support: Wald's contribution was a method for estimating vulnerability from survivor damage.

Three things to keep

1

Count the blanks in every column and turn the count into a percentage: 1% missing and 40% missing need different plans.

Measure the Percentage

Counting isn't enough — you need to know the scale of the damage.

```
df.isnull().sum() / len(df)
```

< 1% missing

Barely a whisper — safe to drop or fill

~40% missing

Systematic absence — rethink your strategy

2

Fill numbers with the middle value unless you have a better reason; fill categories with the most common one; write the choice down.

Every imputation is a guess.

You're not recovering lost truth — you're making an assumption. Document every choice.

What to log

Column filled, method used, reason chosen

Why it matters

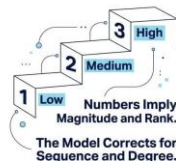
Your model must be reproducible and defensible

3

Unordered categories get one column each; only ordered categories (low, medium, high) become 1, 2, 3.

V30, page 8 of 19 Every imputation is a guess

Label Encoding



Use only when order is lost
1 Low Medium High
2 Medium High Low
3 High Low Medium

V30, page 14 of 19 Label encoding

Question 1 of 3

A retailer's customer table has 10,000 rows. 'Age' is blank in 120 rows and 'Income' is blank in 4,100 rows. What should the analyst do first?

- A** Delete every row that has a blank anywhere.
- B** Fill both columns with the average and move on.
- C** Work out the share of blanks per column (about 1% and 41%) and ask why income is missing so often.
- D** Drop the age column, because it has blanks.

Type the letter in the Zoom chat. You have about 45 seconds. The answer is discussed in the session.

Step 1: See the Damage



Run this first
`df.isnull().sum()`
Counts missing values per column. Zero everywhere? You're lucky. Most of us see numbers.

V30, page 3 of 19 Step 1: see the damage

Measure the Percentage

Counting isn't enough — you need to know the scale of the damage.

```
df.isnull().sum() / len(df)
```

< 1% missing

Barely a whisper — safe to drop or fill

~40% missing

Systematic absence — rethink your strategy

V30, page 4 of 19 Measure the percentage

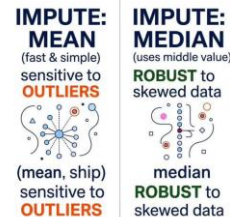
Question 2 of 3

A bank's 'Monthly income' column has 3% of its values missing. Most customers earn between 3,000 and 6,000 a month, but a few earn more than 200,000. Which fill value keeps the column honest?

- A** The average (mean) of the known incomes.
- B** The middle value (median) of the known incomes.
- C** Zero.
- D** The largest known income.

Type the letter in the Zoom chat. You have about 45 seconds. The answer is discussed in the session.

Imputing Numeric Columns



Which to choose?
Mean: Fast - but sensitive to skewed data
Median: Robust - but slower. Outliers often in trouble

17. Rule of Thumb: If your data has outliers, use the median.

V30, page 6 of 19 Imputing numeric columns

Live Example: Customer Dataset

Fill numeric

```
df['Age'].fillna(  
    df['Age'].median(),  
    inplace=True  
)
```

Verify the result

```
df.describe()
```

Spot-check: no NaNs remain. Distributions look reasonable.
Only then move on.

Fill categorical

```
df['Product_Category'].fillna(  
    df['Product_Category'].mode()[0],  
    inplace=True  
)
```

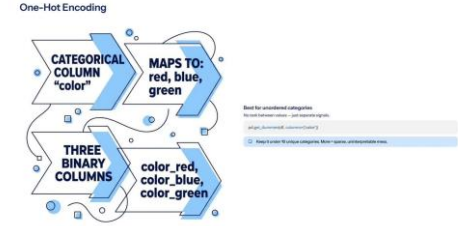
V30, page 9 of 19 Live example: customer dataset

Question 3 of 3

A delivery company wants a model to use 'Region' (North, South, East, West) and 'Service tier' (Basic, Standard, Premium). Which encoding plan is right?

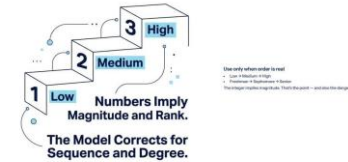
- A** Numbers 1 to 4 for the regions and 1 to 3 for the tiers.
- B** One 0/1 column per region, and 1 to 3 for the tiers because they are ordered.
- C** One 0/1 column per tier, and 1 to 4 for the regions.
- D** Leave both columns as text; the model will manage.

Type the letter in the Zoom chat. You have about 45 seconds. The answer is discussed in the session.



V30, page 12 of 19 One-hot encoding

Label Encoding



V30, page 14 of 19 Label encoding



V30, page 15 of 19 The label encoding trap

Your turn (interactive in the HTML edition)

Fill the gap, then encode the city

Ten customers. Seven have an age; three cells are blank. The known ages are 23, 27, 31, 34, 38, 41 and 45. Choose how the blanks are filled and read what the model will see.

What you can move, and what each control stands for

Fill the blanks with (the middle value (median) / the average (mean) / nothing: leave those rows out)

Chooses the stand-in value written into the three blank cells: the middle value, the average, or none (those three customers are left out).

One customer's age is 92 instead of 45 (on / off)

Replaces one known age (45) with an extreme one (92) and shows which fill value an outlier drags along.

Encode 'City' (Montreal, Toronto, Vancouver) as (numbers 1, 2, 3 / one column per city (0 or 1))

Chooses how the city names become numbers: one number per city (which implies an order) or one 0/1 column per city (no order).

Try, in this order

1. Switch the fill from the middle value to the average, then tick the 92-year-old. Watch which fill value moves.
2. Choose 'leave those rows out'. Count how many customers the model still learns from.
3. Switch the city encoding. Read what the model is told about Vancouver.

The point Every fill is a guess; the middle value is the safest guess. Numbers for categories must not invent an order.

Three things from this block

1

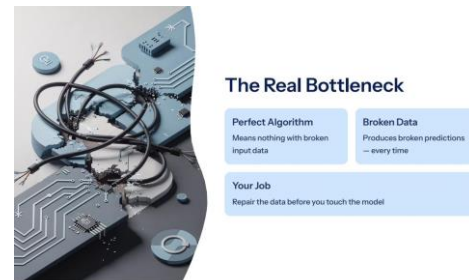
Broken data makes broken predictions, whatever the algorithm.

2

Every fill is a documented guess: which column, which method, why.

3

Words become numbers by encoding: one column per unordered category, ranks only for real orders.



V30, page 2 of 19 The real bottleneck

Every imputation is a guess.

You're not recovering lost truth — you're making an assumption. Document every choice.

What to log

Column filled, method used, reason chosen

Why it matters

Your model must be reproducible and defensible

V30, page 8 of 19 Every imputation is a guess



V30, page 18 of 19 Two obstacles cleared

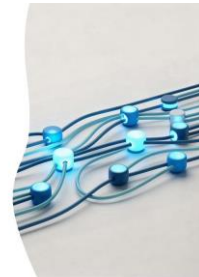
Splitting data and fitting models

Judge a model only on rows it has never seen, write down the shuffle so the split can be repeated, and drive every engine with the same three steps.

The shape of every block: the one idea, a story, three things to keep, three questions, your turn, three things from the block.

Splitting Data & Fitting Models

How to build models that **actually** work on new data



V31, page 1 of 25 Splitting data and fitting models



The Simple Fix: Separate Before You Train

Training Set

Teaches the model

Test Set

Measures honest performance on unseen examples

V31, page 5 of 25 The simple fix: separate before

The Universal Workflow



V31, page 11 of 25 The universal workflow

The one idea

Split before you train: the model learns from one part of the table and is judged on the other, and the split is written down so anyone can repeat it.

Scored on the rows it learned from

1,000 customers. The model learns from all 1,000 and is scored on the same 1,000: 95% right.

It has already seen every answer. The number says little about next month's customers.

the word: training set

Scored on rows it has never seen

700 rows for learning, 300 kept aside. Scored on the 300: 84% right.

That is the number you will get with new customers. Lock the shuffle with a seed (random_state = 42) so the 300 are the same 300 every time.

the word: test set

Split first, fix the seed, then create, fit, predict: the same three steps for every algorithm.



V31, page 5 of 25 The simple fix: separate before

Why Reproducibility Matters

random_state=42
Same seed → same split, every single run

Teammates agree
Anyone running your code gets identical results

6 months later
Re-run: Same output. No surprises.



V31, page 7 of 25 Why reproducibility matters

The Universal Workflow



V31, page 11 of 25 The universal workflow

The story

The lottery that was not random

Step 1

Washington, 1 December 1969

To decide who would be called up first for the Vietnam War, the United States drew birthdays out of a glass container on live television: 366 capsules, one for every day of the year.

Step 2

How the capsules went in

The capsules had been prepared month by month and put into a box in that order, January first, December last. The box was mixed only briefly before the capsules went into the container.

Step 3

The result

November and December birthdays were drawn early far more often than chance allows; the later a month had gone into the box, the sooner it tended to come out.

Step 4

The finding

Within weeks statisticians showed the order was not random. The next lottery, in July 1970, used random tables prepared by the National Bureau of Standards and two rotating drums.

Step 5

For our data

A split is a lottery for your rows. Shuffle properly, write down the seed, and the same rows land in the same sets whenever someone reruns your work.

Retold from the record of the 1969 draft lottery (Selective Service, 1 December 1969) and the statistical analyses published in 1970 and 1971.

Three things to keep

1

Split before you fit; a model scored on the rows it learned from is telling you what it memorised.

2

A typical split keeps 20% to 30% of the rows for the test; write the seed down so the split can be repeated.

3

Create, fit, predict: the same three steps for logistic regression, decision trees and the rest.



V31, page 5 of 25 The simple fix: separate before

Choosing Your Split Ratio



V31, page 9 of 25 Choosing your split ratio

The Universal Workflow



V31, page 11 of 25 The universal workflow

Question 1 of 3

A team trains a churn model on all 2,000 customers, checks it on the same 2,000, and reports 96%. What does the number tell the business?

- A** The model will be right about 96% of the time on next month's customers.
- B** Very little: the model was checked on rows it had already seen.
- C** The model is overfitting by exactly 4%.
- D** The data contains 4% errors.

Type the letter in the Zoom chat. You have about 45 seconds. The answer is discussed in the session.



The 95% Trap

You score 95%
Trained and tested on the
same data

Looks great
But the model is just
memorizing patterns

Fails in the real world
New data exposes the truth

V31, page 2 of 25 The 95% trap



The Real World Doesn't Wait

Customer profiles change. Markets shift. A model that memorized
yesterday's patterns will fail when it matters most.

V31, page 3 of 25 The real world doesn't wait

Question 2 of 3

Two analysts split the same 1,000-row table 70/30 and get different test scores: 84% and 81%. Their code is identical except one line. Which line?

- A** The line that sets the seed (`random_state`).
- B** The line that creates the model.
- C** The line that reads the file.
- D** The line that prints the result.

Type the letter in the Zoom chat. You have about 45 seconds. The answer is discussed in the session.

Why Reproducibility Matters

`random_state=42`

Same seed → same split, every single run

Teammates agree

Anyone running your code gets identical results

6 months later

Re-run. Same output. No surprises.



V31, page 7 of 25 Why reproducibility matters

Without a Seed

Every run shuffles differently

Metrics shift each time

Can't explain your results

⚠ Randomness makes your model impossible to defend. Always set `random_state`.

V31, page 8 of 25 Without a seed

What Happens When You Change `random_state`?

`random_state=42`

Split A → Results A

`random_state=43`

Split B → Results B

Different rows go to training. Different rows to testing. Metrics shift. This is exactly why reproducibility is non-negotiable.

V31, page 21 of 25 What happens when you change `random_state`?

Question 3 of 3

A small clinic has 400 patient records. The analyst proposes a 50/50 split so that the test 'is as big as possible'. What is the cost of that choice?

- A** None; a bigger test set is always better.
- B** The model learns from only 200 records and may learn less than it could.
- C** The test set becomes unreliable.
- D** The seed stops working.

Choosing Your Split Ratio



V31, page 9 of 25 Choosing your split ratio

What Happens When You Change the Ratio?

70 / 30 → Baseline Balanced learning and evaluation	60 / 40 → More Testing Fewer training examples → potential performance drop
--	--

Different ratios affect learning. Experiment deliberately.

Type the letter in the Zoom chat. You have about 45 seconds. The answer is discussed in the session.

V31, page 22 of 25 What happens when you change the ratio?

Your turn (interactive in the HTML edition)

One split, two engines

1,000 telecom customers. Choose the split, the seed and the engine, and read the test score. The scores are fixed values for this exercise, chosen to behave like real runs.

What you can move, and what each control stands for

Split ratio (training / test) (60 / 40 / 70 / 30 / 80 / 20)

How many of the 1,000 rows the model learns from, and how many are kept aside to test it.

Seed (random_state) (42 / 43 / 7)

The seed of the shuffle: which particular rows land in the test set. Same seed, same rows, same score.

Engine (logistic regression / decision tree)

The algorithm fitted on the same training rows: a straight-line scorer, or a chain of yes/no rules.

Score on the training rows instead of the test rows (on / off)

Scores the model on the rows it learned from instead of the test rows: the number it would report if nothing had been kept aside.

Try, in this order

1. Keep everything else and change the seed: how many points does the score move?
2. Switch the engine: is that a bigger or a smaller move than the seed?
3. Tick 'score on the training rows': read the decision tree's number and say why it is meaningless.

The point The split decides the number, so write it down. The engine is one line of code: compare two, and keep the honest test.

Three things from this block

1

Train on one part, test on the other, always.

2

`random_state` locks the shuffle so the split can be reproduced.

3

Create, fit, predict: swapping algorithms is one line.

Four Takeaways



V31, page 23 of 25 Four takeaways

Same Workflow. Different Engine.



V31, page 14 of 25 Same workflow, different engine



V31, page 10 of 25 Scikit-learn's superpower: one API, every algorithm

Reading and reporting model performance

One accuracy number can hide a model that never catches the rare cases: read each class row, and check how many examples stand behind it.

The shape of every block: the one idea, a story, three things to keep, three questions, your turn, three things from the block.

Reading & Reporting Model Performance

Why a single accuracy number can hide catastrophic failures -- and what to do instead.



V32, page 1 of 15 Reading and reporting model



Tool #1: Classification Report

Each Row = One Class
No more aggregate hiding

Each Column = One Metric
Precision Recall F1

Full Visibility
Per-class truth, not averages

V32, page 5 of 15 Tool 1: classification report

Tool #2: Confusion Matrix



What You Get

- Exact predictions vs. reality
- Error patterns jump out visually
- Two lines of code with scikit-learn

Dark cells = common. Light cells = rare. The heatmap tells the story instantly.

V32, page 8 of 15 Tool 2: confusion matrix

The one idea

One overall score can hide a model that is blind to the cases that matter; read the row for each class, and check how many examples stand behind it.

The one number

1,000 patients: 950 healthy, 50 with the disease.

A model that says 'healthy' to everyone is right 950 times: 95% accuracy.

It never found a single patient.

the word: accuracy

The row that matters

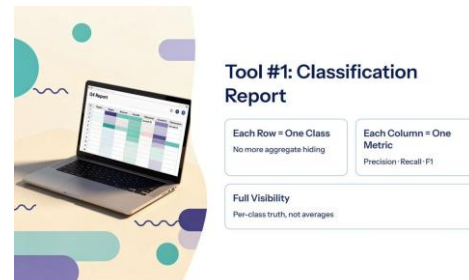
The disease row: 50 cases, 0 caught. Recall 0%.

The support column says 50: so few examples that the overall numbers barely feel them.

Judge that row on its own terms.

the words: recall, support

Read every class row, then save the report next to the model with the same version number.



V32, page 5 of 15 Tool 1: classification report

The Support Column Changes Everything

What Support Tells You

How many test examples belong to each class

- Class 0 → 5,000 samples
- Class 1 → 50 samples

Why It Matters

Imbalanced support means your weighted average is almost entirely driven by the majority class.

A high F1 built on 50 samples is a warning sign — not a green light.

V32, page 6 of 15 The support column changes



V32, page 9 of 15 Real-world example: medical diagnosis

The story

Berkeley, 1973: the number that pointed the wrong way

Step 1

Autumn 1973

The University of California, Berkeley looked at its graduate admissions: about 44% of the men who applied were admitted, and about 35% of the women. It looked like clear bias against women.

Step 2

One number, one story

The overall figures were real, and the gap was large. The university expected to be sued.

Step 3

Department by department

Statisticians split the same applications by department. In most departments, women were admitted at the same rate as men, or at a higher one.

Step 4

The hidden variable

Women had applied in larger numbers to the departments with few places and low admission rates; men to departments that admitted most applicants. The overall number had mixed the two.

Step 5

For our model

An overall score mixes the easy majority with the rare cases that matter. Read the row for each class, and check how many cases stand behind each row, before trusting any average.

Retold from the analysis of the fall 1973 Berkeley admissions published in Science in 1975 (Bickel, Hammel and O'Connell).

Three things to keep

1

Accuracy alone can hide a model blind to the rare class; ask 'how many of the actual cases did it catch?'

2

The support column says how many test examples stand behind each row; a rare class with 50 examples barely moves the overall averages.

3

Save the report with the same version number as the model, so any later question can be answered from the record.

The Follow-Up Question That Changes Everything

How many actual disease cases did it catch?

Recall on Disease Class	Confusion Matrix Truth
0%	
Zero patients correctly identified.	• 50 false negatives — all missed • 0 true positives — none caught • Accuracy was a false confidence signal

V32, page 10 of 15 The follow-up question that

The Support Column Changes Everything

What Support Tells You	Why It Matters
How many test examples belong to each class • Class 0 → 5,000 samples • Class 1 → 50 samples	Imbalanced support means your weighted average is almost entirely driven by the majority class. A high F1 built on 50 samples is a warning sign — not a green light.

V32, page 6 of 15 The support column changes

Tool #3 & #4: Save & Version Everything

 Pair your files model_v2.pkl → evaluation_v2.csv	 Log every batch Track metrics over time in production
 Stay auditable Answer "why did the model do this?" with evidence	



V32, page 12 of 15 Tools 3 and 4: save and version everything

Question 1 of 3

A fraud model reports 98% accuracy on 10,000 transactions, 200 of which are fraudulent. The fraud row of the report shows recall 0%. What happened?

- A** The model caught most of the fraud but raised many false alarms.
- B** The model flags nothing: it is right on the 9,800 honest transactions and wrong on all 200 fraudulent ones.
- C** The data must contain errors.
- D** 98% is a good result for fraud.

Type the letter in the Zoom chat. You have about 45 seconds. The answer is discussed in the session.

92% Accuracy. Sounds Great.

It's a lie.

Deploy it — and users notice it *never* catches fraud. The number looked good. The model wasn't.



V32, page 2 of 15 92% accuracy: sounds great

The Follow-Up Question That Changes Everything

How many actual disease cases did it catch?

Recall on Disease Class	Confusion Matrix Truth
0%	• 50 false negatives — all missed • 0 true positives — none caught • Accuracy was a false confidence signal
Zero patients correctly identified.	

V32, page 10 of 15 The follow-up question that

Minority Class: Judge It Alone

Don't fold it into averages	50 samples = fragile signal	Use both tools together
Evaluate the minority class row on its own terms — never as part of a weighted aggregate.	A strong overall score built on thin minority support is a warning, not validation.	Classification report + confusion matrix = no blind spots.

V32, page 11 of 15 Minority class: judge it alone

Question 2 of 3

Two models both score 92% overall. Model A: majority class 99% right, rare class 10% right. Model B: majority class 90% right, rare class 85% right. The rare class is 'the patient has the disease'. Which do you deploy, and what do you read first?

- A** Model A; it is better on the big class.
- B** Model B; read the disease row first: it catches 85 cases in 100 instead of 10.
- C** Either; they have the same accuracy.
- D** Neither; 92% is too low.

Type the letter in the Zoom chat. You have about 45 seconds. The answer is discussed in the session.

One Number, Two Broken Models

Model A	Model B
Dominates the majority class. Blind to rare cases.	Balanced – but unreliable everywhere.

Same accuracy. Completely different failure modes. You can't tell them apart without digging deeper.

V32, page 3 of 15 One number, two broken models



Tool #1: Classification Report

Each Row = One Class No more aggregate hiding	Each Column = One Metric Precision Recall F1
Full Visibility Per-class truths, not averages	

V32, page 5 of 15 Tool 1: classification report

Minority Class: Judge It Alone

Don't fold it into averages Evaluate the minority class row on its own terms – never as part of a weighted aggregate.	50 samples = fragile signal A strong overall score built on thin minority support is a warning, not validation.	Use both tools together Classification report + confusion matrix = no blind spots.
--	--	---

V32, page 11 of 15 Minority class: judge it alone

Question 3 of 3

A model's weighted-average F1 is 0.93. The support column shows 5,000 examples for class 0 and 40 for class 1. What should the analyst do before reporting 0.93?

- A** Report it; 0.93 is strong.
- B** Read the class-1 row on its own; 40 examples barely move a weighted average.
- C** Remove class 1 from the report to simplify it.
- D** Retrain with a different seed.

Type the letter in the Zoom chat. You have about 45 seconds. The answer is discussed in the session.

The Support Column Changes Everything

What Support Tells You

How many test examples belong to each class

- Class 0 → 5,000 samples
- Class 1 → 50 samples

Why It Matters

Imbalanced support means your weighted average is almost entirely driven by the majority class.

A high F1 built on 50 samples is a warning sign — not a green light.

V32, page 6 of 15 The support column changes

The Weighted Average Trap

High avg + low support = red flag

The weighted average F1 looks strong. But if the minority class has tiny support, that number is lying to you. Always check individual class rows first.

V32, page 7 of 15 The weighted average trap

The Non-Negotiable Checklist

- ☐ Check support before trusting any average
- ☐ Read every class row individually
- ☐ Visualize errors with the confusion matrix
- ☐ Save versioned evaluation artifacts

V32, page 14 of 15 The non-negotiable checklist

Your turn (interactive in the HTML edition)

Read the report yourself

1,000 patients in the test set: 950 healthy, 50 with the disease. Move the two sliders and watch which numbers move. The support of the disease row stays 50 whatever you do.

What you can move, and what each control stands for

Disease cases the model catches (out of 50) (0 to 50, starts at 0)

How many of the 50 sick patients the model flags as sick. It moves the disease row's recall by 2 points per patient.

Healthy patients wrongly flagged (out of 950) (0 to 100, starts at 0)

How many of the 950 healthy patients the model wrongly flags. It lowers precision and takes a tenth of a point off accuracy per patient.

Try, in this order

1. Leave both sliders at 0 and read the accuracy.
2. Move 'cases caught' to 40: how much did accuracy move, and how much did recall move?
3. Now add 60 wrongly flagged patients: which numbers fall, and which barely change?

The point Accuracy moves in tiny steps because it counts all 1,000 patients; the disease row is where the truth lives.

Three things from this block

1

One number hides where a model fails; the class rows show it.

One Number, Two Broken Models

Model A	Model B
Dominates the majority class. Blind to rare cases.	Balanced – but unreliable everywhere.

Same accuracy. Completely different failure modes. You can't tell them apart without digging deeper.

V32, page 3 of 15 One number, two broken models

2

Support tells you how much evidence stands behind a row.

The Support Column Changes Everything

What Support Tells You	Why It Matters
How many test examples belong to each class. • Class 0 → 5,000 samples • Class 1 → 50 samples	Imbalanced support means your weighted average is almost entirely driven by the majority class. A high F1 built on 50 samples is a warning sign — not a green light.

V32, page 6 of 15 The support column changes

3

Version the report with the model; reproducibility is a habit.

Reproducibility Is a Habit

Export classification_report as a dict or CSV. Version it alongside your model. Every. Single. Time.

When a stakeholder asks why your model made a decision — you can **show the exact evaluation that validated it.**

V32, page 13 of 15 Reproducibility is a habit

Communicating results to stakeholders

A result is communicated when the listener can decide: translate the scores into counts, money and a comparison with what happens today.

The shape of every block: the one idea, a story, three things to keep, three questions, your turn, three things from the block.



Communicating Model Results to Stakeholders

From lab metrics to decisions that get deployed.

V33, page 1 of 17 Communicating model results to

Build the Baseline Comparison

Metric	Current Process	Your Model
Customers flagged	80	173
At-risk customers caught	27	237
Miss rate	66%	21%

Same business units, side by side. The question shifts from "Is it good?" to "Is it worth deploying?"

V33, page 6 of 17 Build the baseline comparison

The Translation That Wins the Room

Technical results

- Precision: 0.87
- Recall: 0.79
- AUC: 0.83

Nobody cares.

What you say instead

- Identifies 79% of churners — 237/month vs. 80 today
- Right 87% of the time when it flags someone
- 13% false alarm rate — manageable workload

V33, page 12 of 17 The translation that wins the room

The one idea

A result is communicated when the listener can decide: counts of customers, dollars, and a comparison with today's process.

What the analyst says

Precision 0.87, recall 0.79, AUC 0.83.

Solid scores. The room hears nothing it can act on.

the words: precision, recall

What the room can decide on

300 customers leave every month. The model finds 237 of them; the manual rules find 27.

Of every 100 customers the model flags, 87 really are about to leave.

Is the extra outreach worth 210 extra saves?

the word: baseline

Business words, a baseline comparison, the failure modes named: that is a conversation, not a presentation.

Stop Saying This

✗ Technical jargon

"Precision 0.84, recall 0.81"

Eyes glaze over. Meaning lost.

✓ Business language

"We catch 84% of fraud — but flag 81% of good transactions as false alarms."

Same numbers. Completely different impact.

V33, page 4 of 17 Stop saying this

Build the Baseline Comparison

Metric	Current Process	Your Model
Customers flagged	80	173
At-risk customers caught	27	237
Miss rate	66%	21%

Same business units, side by side. The question shifts from "Is it good?" to "Is it worth deploying?"

V33, page 6 of 17 Build the baseline comparison

The Translation That Wins the Room

Technical results

- Precision: 0.87
- Recall: 0.79
- AUC: 0.83

Nobody cares.

What you say instead

- Identifies 79% of churners — 237/month vs. 80 today
- Right 87% of the time when it flags someone
- 13% false alarm rate — manageable workload

V33, page 12 of 17 The translation that wins the room

The story

Florence Nightingale's diagram

Step 1

Scutari, 1854 to 1856

Florence Nightingale ran the nursing in the British army hospitals of the Crimean War. Soldiers were dying in the wards in far greater numbers than on the battlefield.

Step 2

The counts

She recorded the deaths month by month and by cause: wounds, preventable diseases such as cholera and typhus, and other causes. The numbers were unarguable.

Step 3

The problem with a table

Back in London she had to convince ministers and generals, people who would not read columns of figures and had heard soldiers' deaths described as unavoidable.

Step 4

The picture, 1858

She drew her 'rose' diagram: one wedge per month, its size showing deaths by cause. Preventable disease dwarfed wounds. Anyone could read it in a minute, and sanitary reform followed.

Step 5

For our results

Her numbers were true before the picture; they became a decision only when translated for the listener. Counts per month, a comparison, the cause named: that is what the VP needs from a model report.

Three things to keep

1

Say 'we catch 79 of every 100 leavers', not 'recall 0.79'.

2

Put the model next to the baseline in the same units: flagged, caught, missed, cost of a miss, cost of a false alarm.

3

Answer the two questions before they are asked: how confident are you, and what happens when it is wrong?

Stop Saying This

✗ Technical jargon	✓ Business language
"Precision 0.94, recall 0.91"	"We catch 94% of fraud – but flag 6% of good transactions as false alarms."
Eyes glaze over. Meaning lost.	Same numbers. Completely different impact.

V33, page 4 of 17 Stop saying this

Build the Baseline Comparison

Metric	Current Process	Your Model
Customers flagged	80	173
At-risk customers caught	27	237
Miss rate	66%	21%

Same business units, side by side. The question shifts from "Is it good?" to "Is it worth deploying?"

V33, page 6 of 17 Build the baseline comparison



Two Questions Every Stakeholder Will Ask

- 1 How confident are you?
- 2 What happens when it's wrong?

V33, page 7 of 17 Two questions every stakeholder will ask

Question 1 of 3

The retention director asks: 'So what does the model do for me on Monday?' Which answer lands?

- A 'Precision 0.87 and recall 0.79 on a held-out test set.'
- B 'You get a list of about 272 customers a month; about 237 of them really are about to leave, and we will miss about 63 of the 300 leavers.'
- C 'The AUC is 0.83.'
- D 'It is a gradient boosting model with 200 trees.'

Type the letter in the Zoom chat. You have about 45 seconds. The answer is discussed in the session.

The Translation That Wins the Room

Technical results

- Precision: 0.87
- Recall: 0.79
- AUC: 0.83

Nobody cares.

What you say instead

- Identifies 79% of churners — 237/month vs. 80 today
- Right 87% of the time when it flags someone
- 13% false alarm rate — manageable workload

V33, page 12 of 17 The translation that wins the room

Know Your Audience



CFO

Lead with cost-benefit and downside protection. Skip the metrics.



Department Head

Show the workflow. What changes day-to-day? What gets flagged?



Compliance Team

Show your validation approach and monitoring plan post-deployment.



Skeptical Peer

Show real cases. Before vs. after. Let results speak, not your pitch.

V33, page 14 of 17 Know your audience

Question 2 of 3

A missed leaver costs about \$1,200; every flagged customer gets an \$80 retention offer. The model flags 272 and catches 237 of the 300 leavers; the manual rules flag 80 and catch 27. What does the comparison say?

- A** The model keeps about 210 more customers a month, worth about \$252,000, for about \$15,000 more in retention offers.
- B** The model is too expensive because it flags more people.
- C** The two processes are about equivalent.
- D** They cannot be compared without the AUC.

Type the letter in the Zoom chat. You have about 60 seconds. The answer is discussed in the session.

Build the Baseline Comparison

Metric	Current Process	Your Model
Customers flagged	80	173
At-risk customers caught	27	237
Miss rate	66%	21%

Same business units, side by side. The question shifts from "Is it good?" to "Is it worth deploying?"

V33, page 6 of 17 Build the baseline comparison

Don't Dodge Failure Modes

 False Positive Flag a loyal customer → wasted outreach, ~\$80 retention discount	 False Negative Miss an at-risk customer → lost revenue, ~\$1,200 per churn
---	---

Walk through both. Show the math. This signals respect – and kills the feeling you're hiding something.

V33, page 9 of 17 Don't dodge failure modes

The VP's Real Question

"Are 93 extra conversations worth 210 extra customer saves?"

That's the conversation. Let the math speak for itself.

V33, page 13 of 17 The VP's real question

Question 3 of 3

The CFO gives you five minutes. Which opening line?

- A 'Let me start with how the train-test split was done.'
- B 'The model protects about \$250,000 of revenue a month for about \$15,000 in extra offers; its main risk is missing about 60 leavers.'
- C 'The confusion matrix shows the following four cells.'
- D 'We used scikit-learn.'

Type the letter in the Zoom chat. You have about 45 seconds. The answer is discussed in the session.

Know Your Audience



CFO

Lead with cost-benefit and downside protection. Skip the metrics.



Department Head

Show the workflow. What changes day-to-day? What gets flagged?



Compliance Team

Show your validation approach and monitoring plan post-deployment.



Skeptical Peer

Show real cases. Before vs. after. Let results speak, not your pitch.

V33, page 14 of 17 Know your audience

CFO: Lead With Money



Open with this, not model scores

- Revenue protected per month
- Cost of false positives vs. false negatives
- Net return on deployment investment

Model detail fades to background. Money is the anchor.

V33, page 15 of 17 CFO: lead with money

Your turn (interactive in the HTML edition)

The baseline table, in money

300 customers leave every month. The manual rules flag 80 and catch 27 of them. Set the model's recall and precision and the two prices, and read both processes in the same units.

What you can move, and what each control stands for

Model recall: leavers caught, out of 100 (50 to 95%, starts at 79)

Of the 300 leavers, the share the model finds: caught = 300 times recall.

Model precision: correct flags, out of 100 (60 to 95%, starts at 87)

Of the customers the model flags, the share who really are leaving; the rest are false alarms who still receive an offer.

Cost of one lost customer (500 to 2000 \$, starts at 1200)

What one customer who leaves unnoticed costs; it prices every miss.

Cost of one retention offer (20 to 200 \$, starts at 80)

What one retention offer costs; it prices every flag, right or wrong.

Try, in this order

1. Lower precision to 60%: watch the false alarms and the offer spend.
2. Raise the offer cost to \$200: does the model still win?
3. Set recall to 50%: what happens to the missed leavers and to the net result?

The point The VP's question becomes arithmetic once both processes are written in the same units.

Three things from this block

1

Scores become decisions only when translated into counts and money.

2

Always compare with today's process, in the same units.

3

Name the failure modes and their costs yourself; it builds trust.

The Translation That Wins the Room

Technical results

- Precision: 0.87
- Recall: 0.79
- AUC: 0.83

Nobody cares.

What you say instead

- Identifies 79% of churners — 237/month vs. 80 today
- Right 87% of the time when it flags someone
- 13% false alarm rate — manageable workload

V33, page 12 of 17 The translation that wins the

Build the Baseline Comparison

Metric	Current Process	Your Model
Customers flagged	80	173
At-risk customers caught	27	237
Miss rate	66%	21%

Same business units, side by side. The question shifts from "Is it good?" to "Is it worth deploying?"

V33, page 6 of 17 Build the baseline comparison

Don't Dodge Failure Modes

 False Positive Flag a loyal customer → wasted outreach, ~\$80 retention discount	 False Negative Miss an at-risk customer → lost revenue, ~\$1,200 per churn
---	---

Walk through both. Show the math. This signals respect — and kills the feeling you're hiding something.

V33, page 9 of 17 Don't dodge failure modes

Knowing when to call a decision

Two kinds of decisions look alike: how to build the model well belongs to the data scientist; what the model should care about, and what a mistake costs, belongs to you.

The shape of every block: the one idea, a story, three things to keep, three questions, your turn, three things from the block.

Knowing When to Call a Decision

Your model hit 92% accuracy. Now what? The next move separates great managers from great engineers.



V34, page 1 of 15 Knowing when to call a decision



Two Types of Decisions in Every ML Project

Technical Decisions

How to implement a technique

"What's the most sound engineering choice?"

Business Judgment

What the model should value

"What trade-off aligns with our goals?"

V34, page 3 of 15 Two types of decisions in every

The Rule of Thumb

Does answering this require
understanding **business**
consequences?

YES → It's your call
Own it. Delegate here creates accountability gaps.

NO → Bring in your data scientist
Statistical soundness is their domain. Trust it.

V34, page 6 of 15 The rule of thumb

The one idea

Two kinds of decision look alike: how to build the model well is the data scientist's; what a wrong prediction should cost, and where the cut-off sits, is yours.

A technical call: hand it over

Median or carried-forward value for the blank ages? Which setting reduces overfitting?

The answer depends on the data's statistics, not on business goals.

the word: implementation

A business call: own it

An approved applicant who defaults costs ten times a rejected good applicant. Where do we set the cut-off?

The map of trade-offs comes from the analyst; the point on the map is yours.

the word: threshold

The test: does the answer depend on what each kind of error costs the business? If yes, it is your call.



V34, page 3 of 15 Two types of decisions in every

The Rule of Thumb

Does answering this require understanding **business consequences**?

YES → It's your call

Own it. Delegate here creates accountability gaps.

NO → Bring in your data scientist

Statistical soundness is their domain. Trust it.

V34, page 6 of 15 The rule of thumb

Raising the Threshold to 0.7



This isn't a technical optimization — it's a judgment call rooted in your cost structure. Your data scientist hands you the map. You choose where to land.

V34, page 8 of 15 Raising the threshold to 0.7

The story

Apollo 11: the alarm and the two calls

Step 1

20 July 1969

Apollo 11's lunar module was minutes from the surface of the Moon when its computer flashed an alarm code, 1202, that nobody on board had seen in flight.

Step 2

Whose call?

In Houston, flight director Gene Kranz needed an answer in seconds. He turned to the guidance officer, Steve Bales, who was 26 years old.

Step 3

The technical answer

Bales and his back-room engineer Jack Garman had prepared a list of the computer's alarms. 1202 meant the computer was overloaded but dropping low-priority tasks and restarting correctly. Their answer to the technical question: the machine is still sound. Go.

Step 4

The other decision

Whether a working-but-overloaded computer made the landing an acceptable risk was not an engineering question. That decision belonged to the flight director, and he let the landing continue. Armstrong touched down with well under a minute of fuel to spare.

Step 5

For our model

The analyst tells you whether the machine is sound and what each setting does. Whether the trade-off is acceptable for the business is the call only you can make.

Three things to keep

1

Technical: which method, which setting, does it generalise? Bring in the data scientist.

2

Business: what success means, what error rate is acceptable, where the cut-off sits. Own it.

3

The rule: if the answer depends on what a wrong prediction costs, it is a business call.

Technical Decisions — Delegate These

- Loss Function Selection**
Which minimizes error given your data distribution?
- Overfitting Check**
Does the model generalize — or did it memorize training data?
- Imputation Strategy**
Median vs. mode vs. forward-fill — depends on data structure, not business goals.

V34, page 4 of 15 Technical decisions: delegate

Business Calls — Own These

- Define Success**
What retention rate justifies the cost of outreach to 1,000 flagged customers?
- Set Error Tolerance**
Does preventing one fraud loss justify reviewing 50 flagged transactions?
- Asymmetric Costs**
False negatives and false positives rarely hurt equally — you decide the acceptable gap.



V34, page 5 of 15 Business calls: own these

The Rule of Thumb

Does answering this require understanding **business consequences**?

- YES → It's your call**
Own it. Delegate here creates accountability gaps.
- NO → Bring in your data scientist**
Statistical soundness is their domain. Trust it.

V34, page 6 of 15 The rule of thumb

Question 1 of 3

The data scientist asks: 'Should the missing ages be filled with the median or carried forward from the previous record?' Who decides?

- A** The manager; it affects customers.
- B** The data scientist; the answer depends on the data's structure, not on what an error costs.
- C** The CFO.
- D** Nobody; leave the blanks.

Type the letter in the Zoom chat. You have about 45 seconds. The answer is discussed in the session.

Technical Decisions — Delegate These

 Loss Function Selection Which minimizes error given your data distribution?	 Overfitting Check Does the model generalize — or did it memorize training data?	 Imputation Strategy Median vs. mode vs. forward-fill — depends on data structure, not business goals.
---	---	---

V34, page 4 of 15 Technical decisions: delegate these

The Rule of Thumb

Does answering this require understanding **business consequences**?

YES → It's your call Own it. Delegate here creates accountability gaps.	NO → Bring in your data scientist Statistical soundness is their domain. Trust it.
---	--

V34, page 6 of 15 The rule of thumb

Question 2 of 3

The loan model's cut-off is 0.5 by default. An approved applicant who defaults costs about ten times a wrongly rejected good applicant. Raising the cut-off to 0.7 rejects more good applicants and lets fewer defaults through. Whose decision is it, and why?

A

The data scientist's; a cut-off is a parameter.

B

The manager's; it depends on what each error costs the business.

C

The regulator's.

D

Nobody's; the default is always right.

Type the letter in the Zoom chat. You have about 45 seconds. The answer is discussed in the session.

Case Study: The Loan Model



The Setup
Data scientist recommends threshold = 0.5 (default)
But you know: a false negative costs 10x more than a false positive
Approving a defaulter hurts far more than rejecting a good applicant.

V34, page 7 of 15 Case study: the loan model

Raising the Threshold to 0.7



This isn't a technical optimization — it's a judgment call rooted in your cost structure. Your data scientist hands you the map. You choose where to land.

V34, page 8 of 15 Raising the threshold to 0.7

What the Data Scientist Provides



V34, page 9 of 15 What the data scientist provides

Question 3 of 3

A churn model flags 1,000 customers for outreach. The team asks: 'What retention rate would make the campaign worth it?' What kind of question is this?

- A** A technical question about the algorithm.
- B** A business judgement about costs and returns.
- C** A question for the compliance team.
- D** A question the model answers by itself.

Type the letter in the Zoom chat. You have about 45 seconds. The answer is discussed in the session.

Business Calls — Own These

Define Success

What retention rate justifies the cost of outreach to 1,000 flagged customers?

Set Error Tolerance

Does preventing one fraud loss justify reviewing 50 flagged transactions?

Asymmetric Costs

False negatives and false positives rarely hurt equally — you decide the acceptable gap.



V34, page 5 of 15 Business calls: own these

Stop and Reflect

Can you name the last decision your team handed to a data scientist — that was actually a business call?

If yes: reclaim it. If unsure: that's your starting point.

V34, page 12 of 15 Stop and reflect

Your turn (interactive in the HTML edition)

Five cut-offs, one cost table

1,000 loan applications, 100 of which would default. The analyst supplies the table: at each cut-off, how many defaulters get approved and how many good applicants get refused. You supply the two prices. The cheapest row is highlighted.

What you can move, and what each control stands for

Cost of one approved applicant who defaults (2000 to 20000 \$, starts at 10000)

The price of letting a future defaulter through (a false approval); it multiplies the 'defaulters approved' column.

Cost of one good applicant refused (200 to 2000 \$, starts at 1000)

The price of turning away a good applicant (a false refusal); it multiplies the 'good applicants refused' column.

Try, in this order

1. With the starting prices, which cut-off is cheapest?
2. Halve the cost of a default to \$5,000: does the cheapest row move?
3. Put the cost of a refusal at \$2,000: where does the cheapest row go now?

The point The table is the analyst's job. Choosing the row is yours, because only you know the two prices.

Three things from this block

1

Identify the decision type first: implementation, or trade-off.

2

Own the thresholds, the acceptable error rates and the definition of success.

3

The analyst maps the trade-offs; you choose the point on the map.

Key Takeaways

01	02
Identify the decision type first Technical implementation vs. business trade-off	Delegate technical soundness Algorithms, parameters, and model validation belong to your data scientist
03	04
Own the cost structure Error tolerances, thresholds, and success definitions are yours	Close the loop Track outcomes — judgment without measurement is guesswork

V34, page 14 of 15 Key takeaways

Business Calls — Own These

Define Success What retention rate justifies the cost of outreach to 1,000 flagged customers?
Set Error Tolerance Does preventing one fraud loss justify reviewing 50 flagged transactions?
Asymmetric Costs False negatives and false positives rarely hurt equally — you decide the acceptable gap.



V34, page 5 of 15 Business calls: own these



Your Role in the ML Pipeline

You don't need to write the code. You need to know what it should cost to be wrong — and own that answer.

BUSINESS JUDGMENT TECHNICAL EXPERTISE SHARED OUTCOMES

V34, page 15 of 15 Your role in the ML pipeline

Logistic regression: accuracy and explainability

Logistic regression scores a case by adding one weight per input, so every decision can be explained line by line; the price is that it assumes each input pushes in a straight line.

The shape of every block: the one idea, a story, three things to keep, three questions, your turn, three things from the block.

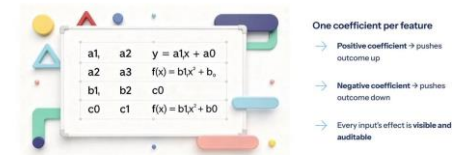
Logistic Regression: Accuracy and Explainability

When a model's decision needs to stand up in court – not just a spreadsheet.



V35, page 1 of 15 Logistic regression: accuracy and

How Logistic Regression Works



V35, page 4 of 15 How logistic regression works

Reading the Coefficients

It's concrete and auditable

Direction
Positive = increases probability
Negative = decreases probability

Magnitude
Income up \$10K Multiply coefficient +
10,000 = change in log-odds

Auditability
Any stakeholder can verify every
factor's contribution



V35, page 13 of 15 Reading the coefficients

The one idea

Logistic regression adds up one weight per input, so a decision can be read line by line; the price is that it assumes every input pushes in a straight line.

A score you can read

Each input has one weight: income pushes the score up by 2 points per \$10,000; each late payment pushes it down by 15.

Add the lines up; that is the score. The loan officer can point to the line that decided.

the word: **coefficient**

The straight-line assumption

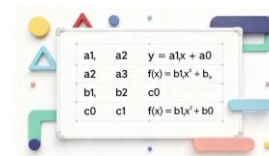
The model assumes every extra \$10,000 of income helps the same amount, from \$20,000 to \$200,000.

If the real effect flattens after \$100,000, the model keeps adding points where the world has stopped.

the word: **linearity**

In lending, health and insurance, a model you can defend line by line is often worth two points of accuracy.

How Logistic Regression Works



- One coefficient per feature
- Positive coefficient → pushes outcome up
 - Negative coefficient → pushes outcome down
 - Every input's effect is visible and auditable

V35, page 4 of 15 How logistic regression works

The Linearity Assumption



- The trade-off
- Assumes a straight-line relationship
 - Real data often curves
 - Feature engineering can help — but adds complexity
 - Simplicity vs. flexibility

V35, page 6 of 15 The linearity assumption

2%

A bank might trade 2% accuracy for full legal protection.

That's not a weakness — that's a rational business decision.

V35, page 12 of 15 2%: a rational business decision

The story

The first points table for lending

Step 1

California, 1956

An engineer, Bill Fair, and a mathematician, Earl Isaac, started a company on a simple claim: lending decisions could be made from a points table computed from data instead of a loan officer's impression.

Step 2

The first scorecard, 1958

They delivered their first credit-scoring system in 1958. Each fact about an applicant earned a number of points; the points were added; the total decided.

Step 3

Why lenders accepted it

A points table is readable. A refused applicant can be told which lines cost the points, and an auditor can check each line. That is exactly the property regulators later demanded.

Step 4

The nationwide score, 1989

Three decades later the company launched a general credit score for the whole market. It still works by adding weighted contributions, and lenders still have to give the reasons for a refusal.

Step 5

For our model

Logistic regression is the modern points table: one weight per input, added up. Readable by design. Its limit: it assumes each input counts the same amount all the way along.

Retold from the company's published history and business press accounts: Fair, Isaac and Company founded in 1956, first scorecard in 1958, general-purpose score in 1989.

Three things to keep

1

One coefficient per input: the sign says the direction, the size says how much.

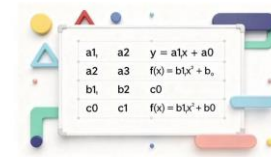
2

Straight lines only: a curved effect (income that stops mattering past \$100,000) is missed unless a feature is added for it.

3

Choose it when the decision must be explained to a regulator, a customer or a clinician, and the data volume is moderate.

How Logistic Regression Works



- One coefficient per feature
- Positive coefficient → pushes outcome up
 - Negative coefficient → pushes outcome down
 - Every input's effect is visible and auditable

V35, page 4 of 15 How logistic regression works

The Linearity Assumption



- The trade-off
- Assumes a straight-line relationship
 - Real data often curves
 - Feature engineering can help — but adds complexity
 - Simplicity vs. flexibility

V35, page 6 of 15 The linearity assumption

When Logistic Regression Wins

- Moderate data volume**
No need for deep learning's massive datasets
- Regulatory scrutiny**
Decisions must be justified to auditors or courts
- Stakeholder understanding**
Non-technical leaders need to trust — and read — the model

V35, page 10 of 15 When logistic regression wins

Question 1 of 3

A bank must give every refused applicant the main reasons for the refusal. Which model makes that easiest?

A

A neural network with hidden layers.

B

Logistic regression: each input has a weight, and the largest negative contributions are the reasons.

C

Gradient boosting with 200 trees.

D

Whichever model has the highest accuracy.

Type the letter in the Zoom chat. You have about 45 seconds. The answer is discussed in the session.

The Problem with Black Boxes

Neural Network Says
Score: 0.23
Reason: Unknown

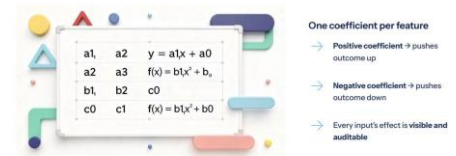
Regulator Asks
"Which factors drove this decision?"

The Gap
Accuracy without explanation isn't enough in regulated industries



V35, page 2 of 15 The problem with black boxes

How Logistic Regression Works



V35, page 4 of 15 How logistic regression works



Credit Scoring

- Explainable Denials**
Regulators and auditors can walk through every coefficient
- Linear Relationships**
Income, debt, and credit history behave roughly linearly
- Legally Defensible**
The model's logic is accountable, not just accurate

V35, page 7 of 15 Use case 1: credit scoring

Question 2 of 3

A retailer's model says every extra \$10,000 of income adds 0.3 to the purchase score. Analysts notice that above \$100,000 income, buying behaviour stops changing. What is the risk?

- A** None; the model adjusts automatically.
- B** The model keeps adding 0.3 per \$10,000 past \$100,000 and overrates high earners; the straight-line assumption is wrong there.
- C** The coefficient turns negative.
- D** The model refuses to run.

Type the letter in the Zoom chat. You have about 45 seconds. The answer is discussed in the session.

The Linearity Assumption



V35, page 6 of 15 The linearity assumption

Reading the Coefficients

It's concrete and auditable

Direction

Positive = increases probability
Negative = decreases probability

Magnitude

Income up \$10K? Multiply coefficient × 10,000 = change in log-odds

Auditability

Any stakeholder can verify every factor's contribution



V35, page 13 of 15 Reading the coefficients

Question 3 of 3

A bank measures logistic regression at 90% and a black-box model at 92%. Regulators must be able to audit every decision. Which choice does the video defend?

- A** The black box; two points is a lot of money.
- B** Logistic regression; two points of accuracy for a model that can be defended line by line is a rational trade.
- C** Neither; wait for a better model.
- D** Use the black box and keep it quiet.

Type the letter in the Zoom chat. You have about 45 seconds. The answer is discussed in the session.



V35, page 11 of 15 The accuracy vs interpretability

2%

A bank might trade 2% accuracy for full legal protection.

That's not a weakness — that's a rational business decision.

V35, page 12 of 15 2%: a rational business decision

Use Logistic Regression When It Matters

- ✓ Interpretability matters
Regulated industry, legal accountability
- ✓ Relationships are linear
Don't add complexity you don't need
- ✓ Stakeholders need to understand
Not just trust — but read — the model

V35, page 15 of 15 Use logistic regression when it matters

Your turn (interactive in the HTML edition)

Read the scorecard

A points table with four inputs. Every applicant starts at 60 points; approval needs 60 or more. Move an input and watch its line change; the total is the sum of the lines.

What you can move, and what each control stands for

Yearly income, k\$ (+2 points per \$10,000) (20 to 200 k\$, starts at 60)

A straight line: every \$10,000 adds the same 2 points, with no bend.

Years at current address (+3 points each) (0 to 15, starts at 4)

Time at one address adds points; stability counts for the lender.

Late payments in two years (-15 points each) (0 to 6, starts at 1)

The largest negative line becomes the stated reason for a refusal.

Existing debt, k\$ (-1 point per \$1,000) (0 to 50 k\$, starts at 10)

Money already owed removes points.

Income effect flattens above \$100,000 (on / off)

Compares the points given with the points deserved if income stops mattering past \$100,000.

Try, in this order

1. Move late payments from 1 to 3: which line changed, and by how much?
2. Raise income to \$180,000 and tick the flattening box: read the gap between the points given and the points deserved.
3. Find the smallest change that flips the decision.

The point Every point is traceable to one line; that is what 'explainable' means. The straight-line assumption is the price.

Three things from this block

1

Coefficients make the model auditable.

2

Linearity keeps it simple and can miss bends.

3

Regulated decision, moderate data, stakeholders who must understand: logistic regression.

Reading the Coefficients

It's concrete and auditable

Direction

Positive = increases probability
Negative = decreases probability

Magnitude

Income up \$10K? Multiply coefficient =
10,000 = change in log-odds

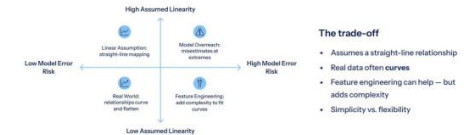
Auditability

Any stakeholder can verify every
factor's contribution



V35, page 13 of 15 Reading the coefficients

The Linearity Assumption



V35, page 6 of 15 The linearity assumption



Use Logistic Regression When It Matters

Interpretability matters

Regulated industry, legal accountability

Relationships are linear

Don't add complexity you don't need

Stakeholders need to understand

Not just trust — but read — the model

V35, page 15 of 15 Use logistic regression when it matters

Decision trees and human reasoning

A decision tree is a chain of yes/no questions you can read aloud; let it grow without limit and it memorises the past, so prune it back to a depth people can follow.

The shape of every block: the one idea, a story, three things to keep, three questions, your turn, three things from the block.



Decision Trees & Human Reasoning

Why interpretability isn't a trade-off — it's a requirement.

V36, page 1 of 18 Decision trees and human

Rules You Can Read Aloud

🟢 If income > \$50,000 AND credit score > 700 → Approve the loan.

Not hidden weights

No mysterious parameters buried in a model.

Explicit rules

Anyone in the room can read, question, and verify.

Plain English

Submittable to compliance teams on day one.

V36, page 5 of 18 Rules you can read aloud

Pruning: Your Main Defense

Cut branches that don't earn their complexity.

Limit tree depth

Cap at 5 levels

Minimum examples per leaf

Require > 20 samples before splitting



V36, page 9 of 18 Pruning: your main defense

The one idea

A decision tree is a chain of yes/no questions you can read aloud; let it grow without limit and it memorises the past, so prune it to a depth people can follow.

Depth 3: rules people can read

'Income above \$50,000? Credit score above 700? Then approve.'

Eight leaves at most; a compliance officer can check each one in an afternoon.

the word: leaf

Depth 12: a wall of rules

Thousands of leaves, 99% right on last year's customers, wrong on new ones: it learned the noise.

Pruning, a depth limit or a minimum of 20 customers per leaf, trades training accuracy for reliability and readability.

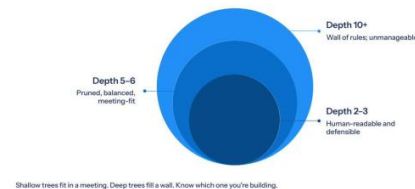
the word: pruning

Shallow trees explain; deep trees memorise; pruning is where you choose.



V36, page 4 of 18 How a decision tree works

Depth = Readability Trade-off



V36, page 6 of 18 Depth = readability trade-off



V36, page 9 of 18 Pruning: your main defense

The story

Twenty questions

Step 1

The parlour game

One player thinks of something; the others may ask up to twenty questions that can only be answered yes or no. A radio version of the game ran for years in the 1940s and 1950s.

Step 2

A good question halves the world

'Is it alive?' cuts the possibilities in half. Twenty halvings: 2 times 2 times 2, twenty times over, is more than a million. Twenty good questions can pick one thing out of a million.

Step 3

A bad question

'Is it my aunt's cat?' rules out one thing. A player who asks such questions is not learning the structure of the world; they are listing things they have seen before.

Step 4

Too many questions

A player allowed two hundred questions could win by listing every object they ever met. Give them an object they have never seen, and the list is useless.

Step 5

For our tree

A decision tree is a game of yes/no questions on the data. Few, good questions: readable rules that hold up on new customers. Hundreds of narrow questions: a memorised list that fails on the next one. Pruning keeps the good questions.

The game is folklore; the arithmetic is exact: two multiplied by itself twenty times is 1,048,576.

Three things to keep

1

Each path from the top to a leaf is a rule in plain words.

2

Trees want to grow until they fit the past perfectly; that is memorising.

3

Prune: limit the depth, require a minimum number of cases per leaf; lose a few points on the past, gain on the future.

Rules You Can Read Aloud

🟢 If income > \$50,000 AND credit score > 700 → Approve the loan.

Not hidden weights

No mysterious parameters buried in a model

Explicit rules

Anyone in the room can read, question, and verify

Plain English

Submittable to compliance teams on day one

V36, page 5 of 18 Rules you can read aloud

The Overfitting Trap

99% Training Accuracy

Tree memorizes every quirk in past data

Fails on New Data

Real-world performance collapses

Fictional Badge

High training score means nothing if prediction breaks



V36, page 7 of 18 The overfitting trap

Pruning: Your Main Defense

Cut branches that don't earn their complexity.

Limit tree depth

Cap at 5 levels

Minimum examples per leaf

Require > 20 samples before splitting



V36, page 9 of 18 Pruning: your main defense

Question 1 of 3

A bank's depth-3 tree approves 85% of good applicants with rules an officer can read aloud. A depth-15 tree approves 91%. A customer complaint arrives. What is the problem with the deep tree?

- A** Nothing; higher approval is better.
- B** Nobody can explain why it said no: higher approval, zero defensibility.
- C** It runs too slowly.
- D** It approves too few applicants.

Type the letter in the Zoom chat. You have about 45 seconds. The answer is discussed in the session.

Case Study: The Shallow Tree

85% Approval Rate

Clear, defensible rules for every decision

Every Denial Explained

Officer can read the reason aloud to the customer

One Afternoon

Compliance team signs off same day

🕒 Interpretability here isn't a concession – it's the **entire point of the model**.

V36, page 13 of 18 Case study: the shallow tree

Case Study: The Deep Tree

91% Approval Rate

Catches more legitimate applicants

Sounds Like Progress

Until a denied applicant files a complaint

Nobody Can Explain It

Compliance team has no answer. Review collapses.

🕒 Higher accuracy. **Zero defensibility**.

V36, page 14 of 18 Case study: the deep tree

Question 2 of 3

Pruning the deep tree to depth 6 gives 88% approval. Its accuracy on the training rows fell from 99% to 88%. Is that a loss?

- A** Yes; eleven points were lost.
- B** No; the 99% was memorised noise. The pruned tree does better on new customers and is readable again.
- C** Yes; a tree should never be pruned.
- D** It cannot be said.

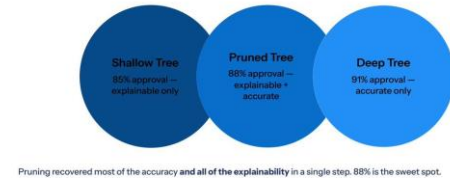
Type the letter in the Zoom chat. You have about 45 seconds. The answer is discussed in the session.

The Accuracy Trade-off



V36, page 10 of 18 The accuracy trade-off

Case Study: The Pruned Tree



V36, page 15 of 18 Case study: the pruned tree

Question 3 of 3

Which of these limits memorising in a tree?

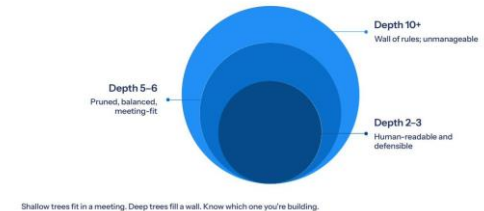
- A** Adding more branches.
- B** Requiring at least 20 customers in a leaf before a split is allowed.
- C** Using more decimal places in the thresholds.
- D** Removing the test set.

Type the letter in the Zoom chat. You have about 45 seconds. The answer is discussed in the session.



V36, page 9 of 18 Pruning: your main defense

Depth = Readability Trade-off



V36, page 6 of 18 Depth = readability trade-off

Your turn (interactive in the HTML edition)

Grow the tree, then prune it

1,000 past customers to learn from, and a set of new customers to check on. Move the depth and watch the number of rules and the two accuracies. The figures are fixed values for this exercise, chosen to behave like a real tree.

What you can move, and what each control stands for

Depth of the tree (questions on the longest path) (1 to 10, starts at 3)

How many yes/no questions the longest rule may ask; each extra level can double the number of rules.

Require at least 20 customers in every leaf (on / off)

Refuses any split that would leave fewer than 20 customers in a leaf, which caps the tree at about 50 rules.

Try, in this order

1. Move the depth from 3 to 10: which accuracy rises and which falls?
2. At depth 10, tick the 20-customer rule: read the number of rules and the new-customer accuracy.
3. Find the depth where the new-customer accuracy is highest.

The point The gap between the two accuracies is memorisation; pruning closes it and keeps the rules readable.

Three things from this block

1

Readable rules are the reason to use a tree.

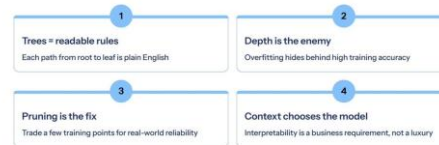
2

Overfitting is the tree's natural tendency.

3

Prune for reliability and readability; the business context decides the depth.

Key Takeaways



V36, page 18 of 18 Key takeaways

**Overfitting =
memorizing noise**

Not learning patterns. The model chases every quirk in historical data – and collapses the moment a new customer walks in.

V36, page 8 of 18 Overfitting = memorizing noise

The Core Trade-off



V36, page 17 of 18 The core trade-off

Gradient boosting

Gradient boosting builds hundreds of small trees one after another, each correcting the mistakes of the ones before; the sum is more accurate than any single tree and no longer readable as rules.

The shape of every block: the one idea, a story, three things to keep, three questions, your turn, three things from the block.

Gradient Boosting

From one tree to 200 – and why that trade-off changes everything.



V37, page 1 of 25 Gradient boosting

How Gradient Boosting Works



V37, page 8 of 25 How gradient boosting works

DECISION FRAMEWORK

When to Use Gradient Boosting

- 1 3,000+ labeled examples
Enough signal for sequential correction to work.
- 2 Accuracy gain justifies opacity
Stakeholders prioritize performance over rule-level explanations.
- 3 Business metric optimization
Revenue, churn, fraud – not regulatory compliance.

V37, page 19 of 25 Decision framework: when to use gradient boosting

The one idea

Hundreds of small trees, built one after another, each correcting the mistakes of the ones before: more accurate than any single tree, and no longer readable as rules.

One tree

'Income below \$50,000 and credit score below 650: deny.' Right 78% of the time. Anyone can read it; a regulator can sign it.

the word: **baseline**

Two hundred trees in sequence

Tree 1 predicts. Tree 2 learns where tree 1 was wrong and corrects. Tree 3 corrects what is left. Add them all up: 92%.

No single rule explains a decision; you see only which inputs matter most.

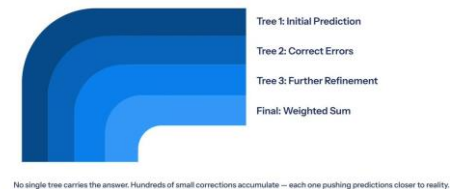
the words: **gradient boosting**

Choose it with 3,000 or more labelled examples, when the accuracy is worth the opacity, and only after a simple baseline has been measured.



V37, page 2 of 25 The 78% problem

How Gradient Boosting Works



V37, page 8 of 25 How gradient boosting works

DECISION FRAMEWORK

When to Use Gradient Boosting

- 1** 3,000+ labeled examples
Enough signal for sequential correction to work.
- 2** Accuracy gain justifies opacity
Stakeholders prioritize performance over rule-level explanations.
- 3** Business metric optimization
Revenue, churn, fraud — not regulatory compliance.

V37, page 19 of 25 Decision framework: when to use gradient boosting

Can weak rules be made strong?

Step 1

A question, 1988

A graduate student, Michael Kearns, asked whether a rule that is only slightly better than a coin flip could be combined with others like it into a rule that is very accurate.

Step 2

The answer, 1990

Robert Schapire proved that it can be done: weak learners can be 'boosted' into a strong one by training each new rule on the cases the previous rules got wrong.

Step 3

A working method, 1995

Freund and Schapire's AdaBoost turned the proof into an algorithm: fit a simple rule, give more weight to its mistakes, fit the next rule, repeat, then add up the votes. It earned them the Gödel Prize in 2003.

Step 4

Gradient boosting, 2001

Jerome Friedman recast the idea: each new small tree fits the error that remains after the sum so far. Later implementations such as XGBoost (2014) made it the default engine for tables of business data.

Step 5

For our choice

Hundreds of small corrections added up: high accuracy, no readable rule. It needs thousands of examples to learn from; below that, the corrections chase noise and a single tree or logistic regression does better.

Three things to keep

1

Sequential: each tree learns from the previous trees' mistakes; a random forest instead builds trees in parallel and averages them.

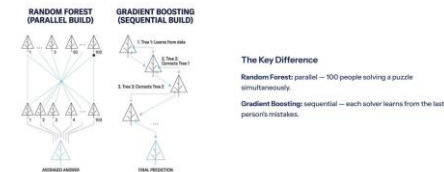
2

It wins on tables of business data with thousands of rows; it needs 3,000 or more labelled examples.

3

Start with a baseline; adopt boosting only if the gain is material and the opacity is acceptable.

How It Differs from Random Forests



V37, page 6 of 25 How it differs from random

Data Thresholds That Matter

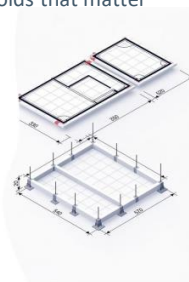
< 500 Examples	500 – 3,000 Examples	3,000+ Examples
Gradient boosting likely performs worse than a simple model. Don't use it.	Gray zone. Test carefully. Consider logistic regression first.	Green light. Gradient boosting has solid ground to stand on.

V37, page 17 of 25 Data thresholds that matter

Always Start With a Baseline

- Step 1: Build logistic regression or a single decision tree.
Step 2: Measure accuracy honestly.
Step 3: Only adopt gradient boosting if the improvement is material.

☐ Did my baseline leave a meaningful gap? If yes – gradient boosting earns its place.



V37, page 21 of 25 Always start with a baseline

Question 1 of 3

A startup with 200 past customers wants to use gradient boosting to predict churn. What does the video predict?

- A** It will work well; boosting is the strongest engine.
- B** It will likely memorise noise and do worse on new customers than logistic regression.
- C** It cannot run on fewer than 1,000 rows.
- D** It will need a GPU.

Type the letter in the Zoom chat. You have about 45 seconds. The answer is discussed in the session.

Data Thresholds That Matter

< 500 Examples

Gradient boosting likely performs **worse** than a simple model. Don't use it.

500 – 3,000 Examples

Gray zone. Test carefully. Consider logistic regression first.

3,000+ Examples

Green light. Gradient boosting has solid ground to stand on.

V37, page 17 of 25 Data thresholds that matter

A Startup Reality Check



200 Historical Customers

Trying to predict churn with gradient boosting?

It will **overfit** — memorizing your 200 cases, not learning real patterns.

A simple logistic regression gives a more **honest picture** of what you actually know.

V37, page 18 of 25 A startup reality check

Question 2 of 3

A fraud model goes from 88% to 93% accuracy with boosting, on 100,000 transactions a year of which 2,000 are fraudulent, at about \$500 per fraud. What is the video's argument for accepting the loss of readable rules?

- A Percentages do not matter; only the mechanism does.
- B The gain is measured in caught cases times \$500, and fraud screening is not a decision that must be explained to each customer.
- C Readable rules never matter anywhere.
- D The regulator will never ask.

Type the letter in the Zoom chat. You have about 45 seconds. The answer is discussed in the session.

Where It Wins in Practice



Fraud Detection

Catch more fraudulent transactions before they clear.



Customer Churn

Identify at-risk customers before they leave.



Sales Forecasting

Predict revenue with structured business data.

V37, page 10 of 25 Where it wins in practice

Accuracy = Real Revenue

2-5%	88→93%	\$500
Accuracy Gain	Fraud Detection	Avg Fraud Value
Typical improvement over baseline on complex business problems	Catches thousands more fraudulent transactions annually	Each percentage point = millions recovered

V37, page 12 of 25 Accuracy = real revenue

When Opacity Is Risky

✓ Acceptable Trade-Off

- Fraud detection
- Churn prediction
- Recommendation engines

⚠ Governance Required

- Consumer lending
- Healthcare diagnosis
- Any regulated decision

In competitive markets, the tradeoff is often acceptable. In regulated industries — you need a strategy.

V37, page 15 of 25 When opacity is risky

Question 3 of 3

A team's logistic regression scores 90% and the business needs 91%. Boosting reaches 92%. What does the video's framework say?

- A** Adopt boosting; it is higher.
- B** Ask whether two points are material for the business and worth the tuning burden and opacity; a simple model that is good enough may be the better choice.
- C** Never use boosting.
- D** Switch to a neural network.

Type the letter in the Zoom chat. You have about 45 seconds. The answer is discussed in the session.

DECISION FRAMEWORK

When to Avoid It

- **Small dataset**
Under 1000 examples —
overfitting risk is high.
- **Regulatory explainability required**
Lending, healthcare, any
decision with legal
accountability.
- **Simple model already works**
If logistic regression hits 90%
and you need 91% — don't add
complexity.

V37, page 20 of 25 Decision framework: when to avoid it

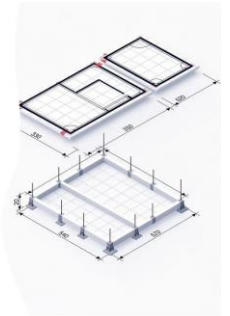
Always Start With a Baseline

Step 1: Build logistic regression or a single decision tree.

Step 2: Measure accuracy honestly.

Step 3: Only adopt gradient boosting if the improvement is **material**.

- ☐ Did my baseline leave a meaningful gap? If yes — gradient boosting earns its place.



V37, page 21 of 25 Always start with a baseline

Your turn (interactive in the HTML edition)

Add trees one at a time

Move the number of trees and read the accuracy on new customers. Change the number of examples available and watch what more trees do with less data. The figures are fixed values for this exercise, chosen to behave like real runs.

What you can move, and what each control stands for

Number of trees (1 to 200, starts at 1)

How many small trees are added one after another; each new tree fits what the previous ones got wrong.

Labelled examples available (5,000 customers / 1,000 customers / 200 customers)

How many labelled customers the trees learn from; with few of them, the later corrections chase noise.

Try, in this order

1. With 5,000 customers, move from 1 tree to 50, then to 200: where does the gain flatten?
2. Switch to 200 customers and move past 20 trees: watch the accuracy fall.
3. Compare the best accuracy reachable with 200 customers and with 5,000.

The point More trees help only when there is enough data to learn from; the gain over the baseline is what justifies the opacity.

Three things from this block

1

Sequential correction is the mechanism; opacity is the cost.

2

Tabular business data with volume: boosting is the default engine.

3

Baseline first; boosting must earn its place.

The Core Mechanism — Summarized



That's the fundamental mechanism — and the fundamental cost. Transparency traded for accuracy, systematically.

V37, page 22 of 25 The core mechanism.

Gradient Boosting Dominates Tabular Data



V37, page 23 of 25 Gradient boosting dominates



The Bottom Line

Use it wisely.

Gradient boosting is not a universal answer. It's a **precision tool** — powerful when conditions align, costly when they don't.

✓ Use When
Volume • accuracy need • tolerance for opacity

✗ Skip When
Small data • regulatory burden • simple baseline works

V37, page 25 of 25 The bottom line

Where neural networks fit

Neural networks are specialists for pictures, sound and text at large scale; on a spreadsheet of customers, a tree-based model is usually faster, cheaper, more accurate and easier to explain.

The shape of every block: the one idea, a story, three things to keep, three questions, your turn, three things from the block.



V38, page 1 of 20 Where neural networks fit

The Fundamental Split

Structured Data

Spreadsheets, CRM records, sales logs

- Rows and columns
- Less data needed
- Tree-based methods win

Unstructured Data

Images, audio, raw text

- High-dimensional inputs
- Features hard to hand-craft
- Neural networks win

V38, page 5 of 20 The fundamental split

Algorithm Selection Framework



Matching algorithm to data type is the highest-leverage decision you'll make on any ML project.

V38, page 16 of 20 Algorithm selection framework

The one idea

Neural networks are specialists for pictures, sound and text at large scale; on a table of customers, a tree-based model is faster, cheaper, at least as accurate, and easier to explain.

Head to head on a table (churn)

Gradient boosting: 5,000 records, 92%, runs in milliseconds on an ordinary computer, inputs can be ranked.

Neural network: 50,000 records, 91%, needs a GPU, cannot easily say which inputs mattered.

the words: structured data

Where the network wins

Photos, audio, raw text: millions of pixels or words and no ready-made columns.

The network builds its own features, layer by layer, and needs enormous data and computing to do it.

the words: deep learning

Match the tool to the shape of the data: rows and columns go to trees; pixels, sound and words go to networks.

The Fundamental Split

Structured Data	Unstructured Data
Spreadsheets, CRM records, sales logs	Images, audio, raw text
• Rows and columns • Less data needed • Tree-based methods win	• High-dimensional inputs • Features hard to hand-craft • Neural networks win

V38, page 5 of 20 The fundamental split

Head-to-Head: Churn Prediction

STRUCTURED	NEURAL NETWORK
• 5,000 training records • 92% accuracy • Runs in milliseconds on CPU • Feature importance – fully explainable	• 50,000 training records required • 91% accuracy • Requires GPU infrastructure • Black box – no easy explanation

V38, page 10 of 20 Head-to-head: churn prediction

Algorithm Selection Framework



Matching algorithm to data type is the highest-leverage decision you'll make on any ML project.

V38, page 16 of 20 Algorithm selection framework

The story

The year the pictures changed

Step 1

A very large picture set, 2009

Researchers led by Fei-Fei Li built ImageNet: millions of photographs, each labelled by hand with what it shows.

Step 2

The contest

From 2010, teams competed to label new photos automatically. In 2011 the best system still got about a quarter of its top-five guesses wrong.

Step 3

2012

A deep neural network from Toronto, built by Krizhevsky, Sutskever and Hinton, cut the error to about 15%. It was trained for days on two graphics cards with 1.2 million labelled images.

Step 4

Why it worked here

A photo has no ready-made columns. The network learned its own features from the pixels, layer by layer. That takes millions of examples and heavy computing, and it had both.

Step 5

For our choice

Give the same machine a spreadsheet of 5,000 customers and it has nothing to build features from and too little data to learn. There the tree-based models win. Networks are specialists, not the default.

Retold from the published results of the 2012 ImageNet challenge and the paper describing the winning network.

Three things to keep

1

Three families: rule-based (trees), ensembles (boosting, forests), deep learning (networks).

2

Structured data favours trees; unstructured data (images, audio, text) favours networks.

3

Networks build their own features, and pay for it with enormous data and computing needs.

Three Algorithm Families



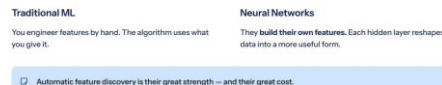
V38, page 4 of 20 Three algorithm families

The Fundamental Split



V38, page 5 of 20 The fundamental split

The Key Difference



V38, page 7 of 20 The key difference

Question 1 of 3

A retailer has 8,000 rows of customer data (age, spend, contract type, calls to support) and wants to predict churn. Which family first?

- A** A deep neural network.
- B** Tree-based models, such as gradient boosting.
- C** A large language model.
- D** A convolutional network.

Type the letter in the Zoom chat. You have about 45 seconds. The answer is discussed in the session.

The Fundamental Split

Structured Data	Unstructured Data
Spreadsheets, CRM records, sales logs	Images, audio, raw text
• Rows and columns • Less data needed • Tree-based methods win	• High-dimensional inputs • Features hard to hand-craft • Neural networks win

V38, page 5 of 20 The fundamental split

Head-to-Head: Churn Prediction

Tree-Based Models	Neural Networks
• 5,000 training records • 92% accuracy • Runs in milliseconds on CPU • Feature importance – fully explainable	• 50,000 training records required • 91% accuracy • Requires GPU infrastructure • Black box – no easy explanation

V38, page 10 of 20 Head-to-head: churn prediction

Algorithm Selection Framework



Matching algorithm to data type is the highest-leverage decision you'll make on any ML project.

V38, page 16 of 20 Algorithm selection framework

Question 2 of 3

A manufacturer wants to spot defective parts from photos taken on the production line and has 300,000 labelled photos. Which family?

- A** Logistic regression on the average colour of each photo.
- B** A neural network built for images (convolutional), which learns its own visual features at this scale.
- C** A decision tree on the file names.
- D** A spreadsheet rule.

Type the letter in the Zoom chat. You have about 45 seconds. The answer is discussed in the session.

When Neural Networks Earn Their Keep



Satellite Imagery

Millions of pixels, patterns impossible to hand-engineer



Audio & Speech

Raw waveforms with complex temporal structure



Raw Text

Language understanding at massive scale

V38, page 9 of 20 When neural networks earn their keep

WHERE NEURAL NETWORKS WIN

Computer Vision Changed Everything



Convolutional neural networks gave machines the ability to see.

- Image classification
- Object detection
- Medical imaging analysis

These tasks are impossible without deep learning.

V38, page 13 of 20 Where neural networks win: computer vision

Question 3 of 3

The startup in the video spent months and GPU money on a network that lost to a tree built in an afternoon. What was the mistake?

- A** Too few layers in the network.
- B** Choosing the tool by its sophistication instead of by the shape and size of the data.
- C** The wrong programming language.
- D** No seed for the split.

Type the letter in the Zoom chat. You have about 45 seconds. The answer is discussed in the session.



V38, page 2 of 20 The \$100K mistake

Fancier $\neq$ Better

The neural network failed not because it was broken – but because it was the wrong tool for the job.

V38, page 3 of 20 Fancier is not better

**Know your tools.
Choose wisely.**

The best algorithm isn't the newest or the most sophisticated.
It's the one that fits your problem.

V38, page 20 of 20 Know your tools, choose wisely

Your turn (interactive in the HTML edition)

Pick the family for the data

Describe the data you have and read the family the video's framework points to, with the reasons.

What you can move, and what each control stands for

What the data looks like (a spreadsheet of customer records / photos of products / recorded customer calls / free-text complaints)

The shape of the data: rows and columns with ready-made features, or pixels, sound and words with none.

How much of it there is (500 rows or files / 5,000 rows or files / 50,000 rows or files / 5 million rows or files)

How many rows or files exist; a network needs very many to learn its own features.

Must each decision be explained to someone? (no / yes)

Whether each decision must be justified to a person; that requirement can veto an opaque model.

Try, in this order

1. A spreadsheet, 5,000 rows, explanations required.
2. Photos, 500 files: read the warning.
3. Photos, 5 million files, no explanation needed.

The point The shape of the data picks the family; the size confirms it; the need to explain can veto it.

Three things from this block

1

Match the algorithm to the data type.

Key Takeaways

- 1 Match algorithm to data type**
Structured → trees.
Unstructured → neural networks.
- 2 Simpler models often win on business data**
Gradient boosting is the workhorse most companies should study first.
- 3 Interpretability has real business value**
If stakeholders need to understand the why, tree-based methods deliver.

2

Neural networks: powerful specialists, expensive generalists.

V38, page 18 of 20 Key takeaways

Neural Networks Are Specialists

- Powerful in Their Domain**
Vision, language, audio — unmatched performance at scale.
- Costly Outside It**
Tabular business data? You're paying more for less.
- Not General-Purpose**
Don't default to them because they sound impressive.

3

For tabular business data, start with trees.

V38, page 17 of 20 Neural networks are specialists

**Know your tools.
Choose wisely.**

The best algorithm isn't the newest or the most sophisticated.
It's the one that fits your problem.

V38, page 20 of 20 Know your tools, choose wisely

Black box vs glass box

Choosing an algorithm is a business decision made with three questions: how much labelled data, must people understand each prediction, and what accuracy does the business need.

The shape of every block: the one idea, a story, three things to keep, three questions, your turn, three things from the block.



Black Box vs. Glass Box

94% accuracy your VP can't explain -- or 87% everyone trusts. Which do you choose?

V39, page 1 of 28 Black box vs glass box

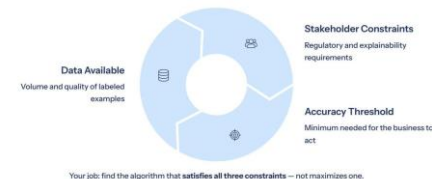
Three Questions Before You Touch Any Algorithm



Get this wrong and no amount of model tuning saves the project in the boardroom.

V39, page 3 of 28 Three questions before you touch

The Three-Constraint Framework



V39, page 21 of 28 The three-constraint framework

The one idea

Choosing an algorithm is a business decision made with three questions: how much labelled data, must people understand each prediction, and what accuracy does the business need.

94% that nobody can explain

The VP cannot justify a budget on it; the sales team will not trust it.

It sits on a shelf and contributes zero.

the words: black box

87% that everyone understands

Adopted on Monday, refined over time.

A feature-importance chart shows the five drivers the marketing team can act on.

the words: feature importance

Interpretability and accuracy are two of three constraints; the third is your data and regulatory environment. Satisfy all three well enough to act.



V39, page 2 of 28 The real problem

Three Questions Before You Touch Any Algorithm



Get these wrong and no amount of model tuning saves the project in the boardroom.

V39, page 3 of 28 Three questions before you touch

The Three-Constraint Framework



V39, page 21 of 28 The three-constraint framework

The story

MYCIN: the program that was right and never used

Step 1

Stanford, early 1970s

Edward Shortliffe built MYCIN, a program that recommended antibiotics for serious blood infections from about 600 rules written with physicians.

Step 2

It could explain itself

Ask MYCIN why, and it showed the chain of rules behind its advice. In a blinded evaluation published in 1979, outside experts rated its recommendations acceptable more often than those of the Stanford faculty physicians.

Step 3

It was never used

The planned trial on the hospital wards never took place. Not one patient was treated on MYCIN's advice in routine care, and its knowledge base was retired in 1978.

Step 4

Why

Entering a case took more than half an hour at a terminal, the program did not fit how doctors worked, and nobody had settled who would answer for it if it was wrong.

Step 5

For our model

Accuracy did not ship it, and even explanations were not enough. A model creates value only when the people who must act on it trust it, can use it in their day, and know who is accountable. That is what decides adoption.

Retold from the evaluation published in JAMA in 1979 (Yu and colleagues) and from Buchanan and Shortliffe's own account of the project (1984).

Three things to keep

1

Ask the three questions before naming an algorithm: how much labelled data, must people understand why, what accuracy bar.

2

Logistic regression for auditable coefficients; trees for rules operators can override; boosting for accuracy plus a feature-importance chart.

3

An unused accurate model is worth zero; trust is what turns a model into an asset.

Three Questions Before You Touch Any Algorithm



Get these wrong and no amount of model tuning saves the project in the boardroom.

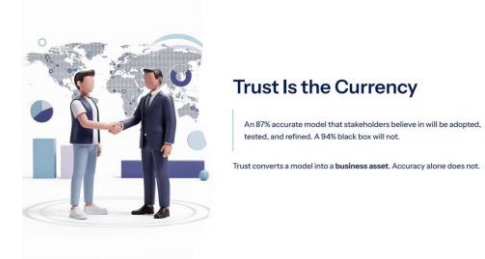
V39, page 3 of 28 Three questions before you touch

Three Algorithms. Three Contexts.



No algorithm wins every context. The right fit depends on your three answers.

V39, page 7 of 28 Three algorithms, three contexts



V39, page 20 of 28 Trust is the currency

Question 1 of 3

A lending model in a regulated market: logistic regression at 82%, gradient boosting at 94%. Which does the video pick, and why?

- A Boosting; twelve points is decisive.
- B Logistic regression; every coefficient can be audited and every refusal explained, and the compliance gap would cost more than the accuracy gap.
- C A decision tree, always.
- D Whichever the data scientist prefers.

Type the letter in the Zoom chat. You have about 45 seconds. The answer is discussed in the session.

Regulation Makes This Non-Negotiable



HEALTHCARE
(patient safety)

FINANCE
(lending
decisions)

INSURANCE
(risk scoring)

The Equal Credit Opportunity Act (US) requires lenders to provide specific reasons for adverse credit decisions.

A gradient boosting model you cannot explain doesn't just underperform — it may be non-compliant and expose the organization to legal liability.

HEALTHCARE
(patient
safety)

FINANCE
(lending
decisions)

INSURANCE
(risk scoring)

V39, page 19 of 28 Regulation makes this non-negotiable



SCENARIO 1

Lending Model in a Regulated Market

Winner: Logistic Regression

Regulators audit every coefficient
Full transparency required by law

82% beats 94% here
The compliance and trust gap costs more than the accuracy gap

V39, page 22 of 28 Scenario 1: lending model in a regulated market

Question 2 of 3

Internal customer segmentation for marketing, no regulation, 20,000 customers. The team needs to know what drives segment membership. Which combination?

- A** Gradient boosting plus a feature-importance chart.
- B** Logistic regression only.
- C** A neural network.
- D** Manual rules written by the team.

Type the letter in the Zoom chat. You have about 45 seconds. The answer is discussed in the session.



V39, page 16 of 28 Gradient boosting in practice: churn prediction for marketing



V39, page 23 of 28 Scenario 2: internal customer segmentation

Question 3 of 3

E-commerce fraud: the compliance team needs rules it can audit, and operators must be able to override the model when they see a new pattern. Which model?

- A** Gradient boosting at 93%, kept in staging until the team trusts it.
- B** A decision tree at 85%: readable if-then rules, deployed now and improved with human feedback.
- C** A neural network.
- D** No model; rules only.

Type the letter in the Zoom chat. You have about 45 seconds. The answer is discussed in the session.

DECISION TREES — IN PRACTICE

Fraud Detection



A rule that flags "card used in two countries within one hour" is:

- Auditable by compliance
- Overridable by human judgment
- Defensible in a regulatory review

Model logic and human expertise work in tandem.

V39, page 11 of 28 Decision trees in practice: fraud detection



SCENARIO 3

Fraud Detection in E-Commerce

Winner: Decision Trees

85% deployed now

Team understands, adjusts, improves through human feedback

93% sits in staging

Team distrusts it — business captures zero value

V39, page 24 of 28 Scenario 3: fraud detection in e-commerce

Your turn (interactive in the HTML edition)

The three-question decision matrix

Answer the three questions of the video and read the algorithm the framework points to, what to show stakeholders, and the caution that applies.

What you can move, and what each control stands for

How many labelled examples do you have? (fewer than 1,000 / 1,000 to 3,000 / more than 3,000)

Question 1: the volume of labelled examples; below a few thousand, gradient boosting memorises noise.

Must each prediction be explained? (yes, to regulators or customers / partly: the main drivers are enough / no)

Question 2: whether regulators, customers or staff must be able to trace each prediction.

What accuracy does the business need? (about 80% is enough / around 90% / as high as possible)

Question 3: the accuracy bar the business needs before it will act on the model.

Try, in this order

1. Choose fewer than 1,000 examples and 'as high as possible': read the caution.
2. Choose 'yes, to regulators': which model appears, whatever the other answers?
3. Choose 'no' and 'more than 3,000': what do you hand the marketing team?

The point The three answers set the constraints before a line of code; the algorithm follows from them.

Three things from this block

1

Algorithm selection is a business decision, not a leaderboard.

2

Three constraints: data, stakeholders and regulation, the accuracy bar.

3

Feature importance is the bridge between a complex model and the people who must act.



V39, page 25 of 28 Algorithm selection is a business

Interpretability vs. Accuracy?

They're not opposites.

They're two constraints in a three-constraint problem. The third is your data and regulatory environment.

Data Environment	Interpretability Needs	Accuracy Threshold
Volume, quality, labeling coverage	Regulatory, stakeholder, customer-facing	Minimum viable for business action

V39, page 26 of 28 Interpretability vs accuracy?

Feature Importance: Your Communication Bridge

Capture accuracy gains from powerful models — without losing the ability to explain and defend your outputs.

- Most business ML projects can use a more complex model than they think — as long as feature importance is part of the communication plan from day one.



V39, page 27 of 28 Feature importance: your communication bridge

Before Thursday

To do before the next session

1. Watch Videos 40 to 50 before Thursday.
2. Finish the Notebook Lab (Module 6). About 90% accuracy with 0% recall on churned customers is the expected result, not a mistake: that is the lesson.
3. Write the Algorithm Selection Briefing Note (Module 7): 400 words, one recommended algorithm, three criteria.
4. Submit the Module 8, 9 and 10 activities before Thursday (Segment Interpretation, Bias Audit Report, LLM Output Audit).

Next session

Thursday 17 September 2026, 11:00

Session 4: learning without labels, bias and governance, how large language models work (Videos 40 to 50)

What is assessed, and when

Quiz 3 (Modules 5 and 6), 15 minutes online	before Thursday 17 September
Notebook Lab and Algorithm Selection Briefing Note	before Thursday 17 September
Midterm exam: Modules 1 to 6, 90 minutes online, 30% of the grade, no coding	between Friday 18 and Wednesday 23 September
Quizzes 4 and 5 (Modules 7 to 10)	before Thursday 24 September
Attendance and participation, 10%	pre-session submissions and contributions in every session